

Script-Controlled ACL - Restrict Record Access Based on Field Value

ServiceNow Micro-Project

PHASE 3 | PROJECT DESIGN PHASE

3. Project Design Phase

3.1 Solution Architecture Overview

The solution has three logical layers that sit on top of the standard ServiceNow platform: an **Identity layer** (users and custom roles), a **Data layer** (the Institution Details table and its records), and a **Security layer** (four script-controlled ACLs that gate read, create, write, and delete operations). Every request to the data layer must pass through the security layer before the platform renders it in a form, list, or portal widget.

Request flow: User Request → ServiceNow Security Engine → Access Control (ACL) evaluation (Requires role → Data Condition → Advanced Script) → Allow (render record) or Deny (no record shown).

3.2 Entity Design — Institution Details Table (u_institution_details)

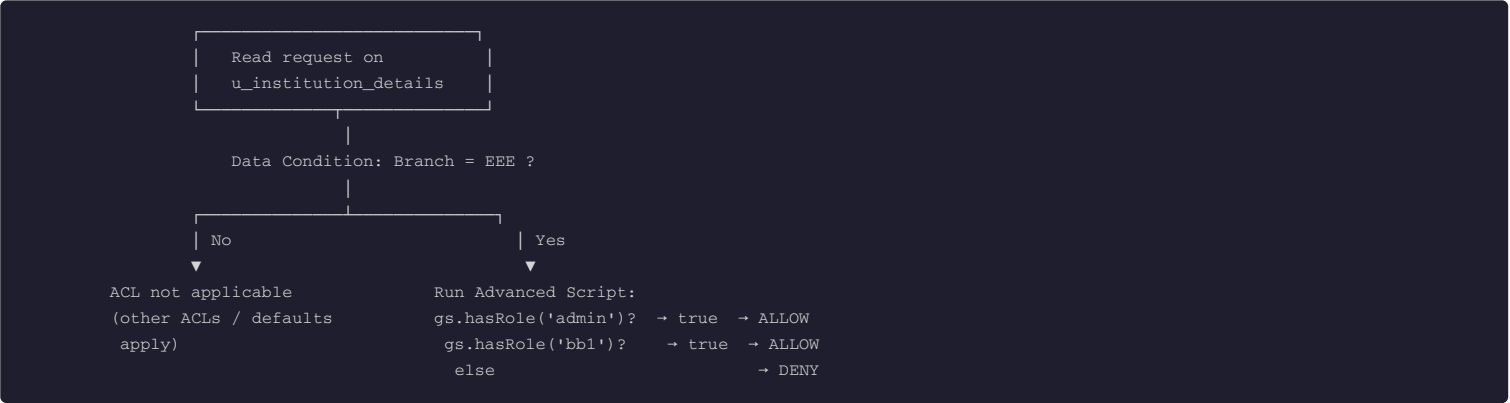
Field Label	Column Name	Type	Notes
Student Roll Number	u_student_roll_number	Auto Number	System-generated unique identifier (e.g., INS0001001)
Student Name	u_student_name	Reference - User	Points to sys_user
Faculty Name	u_faculty_name	Reference - User	Points to sys_user
Branch	u_branch	Choice	Values: ECE, EEE, CSE — the field the ACL script evaluates
Email	u_email	String	Max length 40
Phone Number	u_phone_number	String	Max length 40
Description	u_description	String (multi-line)	Max length 1000

3.3 Role Design

Role	Operation Governed	Description
bb1	Read	Grants read access to EEE-branch records only
bb2	Create	Grants ability to create new records in the table
bb3	Write	Grants ability to edit existing records
bb4	Delete	Grants ability to delete records
admin	All (Read/Create/Write/Delete)	Built-in role; always bypasses the branch restriction

The EEE User is assigned all four custom roles (bb1-bb4) so that the full CRUD lifecycle can be demonstrated for a single test persona, while the branch-level Data Condition still restricts *which rows* are visible.

3.4 ACL Decision Logic (Read ACL)



3.5 ACL Script Design (all four operations)

```
(function () {
  // Allow admin users full access
  if (gs.hasRole('admin')) {
    return true;
  }
  // Allow only the branch-specific role
  if (gs.hasRole('<bb1 | bb2 | bb3 | bb4>')) {
    return true;
  }
  // Deny access for all others
```

```
        return false;
    }) ();
```

The Read ACL additionally carries a **Data Condition** (`Branch is EEE`) so that the script is only consulted for EEE rows; Create/Write/Delete ACLs are role-gated without a data condition, since they act on the record being submitted rather than a pre-existing row set.

3.6 Access Control Matrix

User	Roles Held	Read EEE rows	Read other-branch rows	Create	Write	Delete
Administrator	admin	Allow	Allow	Allow	Allow	Allow
EEE User	bb1, bb2, bb3, bb4	Allow	Deny (not covered by role)	Allow	Allow	Allow
Any user without bb1-bb4	none	Deny	Deny	Deny	Deny	Deny

3.7 UI / Interaction Design

- Verification is performed by using ServiceNow's built-in **Impersonate user** feature rather than building a custom UI, keeping the design minimal per the "script-controlled ACL, not custom UI" scope decided in Phase 1.
- The standard list view (`u_institution_details_list.do`) is used as the primary interface for demonstrating filtered results.
- The standard "Actions on selected rows → Delete" list action is used to demonstrate the Delete ACL, so no custom button/UI Action is required.

3.8 Design Rationale

Separating the four CRUD operations into four distinct ACL records (rather than one combined ACL) keeps each rule single-purpose and easy to audit, mirrors ServiceNow's native ACL model (one ACL per table+operation), and allows roles to be granted incrementally (`bb1 → bb2 → bb3 → bb4`) to demonstrate progressively expanding access during testing.